

*args

Requisitos de finalización

***args** es una convención que se utiliza para permitir que una función acepte un número variable de argumentos posicionales. La palabra **args** es simplemente un nombre convencional, y el asterisco (*) es el que realmente tiene el significado especial. Aquí está cómo funciona y para qué se utiliza:

¿Qué es *args?

Es una forma de definir una función que puede recibir un número arbitrario de argumentos posicionales. Estos argumentos se recopilan en una [tupla](#), lo que permite a la función acceder a ellos dentro de su cuerpo.

Se utiliza cuando no sabes de antemano cuántos argumentos posicionales se pasarán a la función. Esto es útil para [funciones](#) genéricas o cuando los parámetros pueden variar.

¿Cómo se usa *args?

Cuando se define una función con ***args**, cualquier argumento adicional que se pase a la función se recopilará en una [tupla](#) llamada **args** (aunque podrías usar cualquier nombre después del *).

Ejemplo:

```
def suma(a,b):  
    return a+b  
print(suma(5,6,))
```

```
def suma_args(*args):  
    return sum(args)
```

```
def suma_args(*args):
```

```
    # funciona de la misma manera  
    total = 0  
    for arg in args:  
        total += arg  
    return total
```

Atributos

Requisitos de finalización

Se define una clase llamada **Pajaro** con un atributo de instancia llamado **color**. A continuación, se crea una instancia de la clase **Pajaro** con el color "negro" y se imprime el valor del atributo **color** de esa instancia.

- Los atributos de instancia en Python se crean en el constructor `__init__` usando `self`, lo que permite que cada instancia de la clase tenga su propio valor para esos atributos.
- `self.color = color` asigna el valor pasado al crear la instancia a un atributo de instancia llamado `color`.
- Puedes acceder a los atributos de una instancia usando la notación de puntos, como en `mi_pajaro.color`.

Ejemplo:

```
class Pajaro:

    def __init__(self, color):

        self.color = color

    def __str__(self):

        return f"Pajaro de color {self.color}"

mi_pajaro = Pajaro("negro")

print(mi_pajaro)
```

Bool

Requisitos de finalización

El tipo de dato booleano (`bool`) representa dos valores: `True` (verdadero) y `False` (falso). Los valores booleanos son fundamentales para controlar el flujo del [programa](#) mediante declaraciones condicionales y bucles. Además, son el resultado de evaluaciones de expresiones lógicas o comparaciones.

Definición y Uso de Booleanos

Los valores booleanos se pueden crear de manera explícita usando `True` y `False` (notar la T y la F en mayúsculas):

```
es_mayor = True
```

```
es_menor = False
```

Expresiones Booleanas

Una expresión booleana es una expresión que evalúa a `True` o `False`. Python proporciona varios operadores para crear expresiones booleanas:

```
if es_mayor:
```

```
print("Es verdadero.")
```

else:

```
print("Es falso.")
```

Una expresión booleana es una expresión que evalúa a **True** o **False**. Python proporciona varios operadores para crear expresiones booleanas:

Operadores de Comparación: Devuelven **True** o **False** basado en la comparación de dos valores.

```
a = 10
```

```
b = 5
```

```
print(a > b)
```

```
print(a < b)
```

```
print(a == 10)
```

```
print(a != b)
```

Operadores Lógicos: Permiten combinar expresiones booleanas.

- **and:** Devuelve **True** si ambas expresiones son verdaderas.
- **or:** Devuelve **True** si al menos una de las expresiones es verdadera.
- **not:** Invierte el valor booleano de la expresión.

```
x = True
```

```
y = False
```

```
print(x and y)
```

```
print(x or y)
```

```
print(not x)
```

Valores Verdaderos y Falsos en Python

En Python, algunos valores son considerados **True** y otros **False** cuando se evalúan en un contexto booleano:

- **Falsos:** Los siguientes valores son considerados **False**:
 - **None**
 - **False**
 - **0** (cero de cualquier tipo: **0**, **0.0**, **0j**)
 - Secuencias y colecciones vacías: **''** (cadena vacía), **[]** (lista vacía), **()** ([tupla](#) vacía), **{}** (diccionario vacío), **set()** (conjunto vacío), [range\(0\)](#).
- **Verdaderos:** Cualquier valor que no sea considerado **False** se evalúa como **True**.

False:

```
print(bool(0))
```

```
print(bool([]))
```

True:

```
print(bool(1))
```

```
print(bool([1, 2]))
```

Ejemplos:

- **Condicionales:**

```
edad = 18  
es_adulto = edad >= 18
```

```
if es_adulto:
```

```
    print("Eres mayor de edad.")
```

```
else:
```

```
    print("Eres menor de edad.")
```

- **Bucles:**

```
numero = 5
```

```
while numero > 0:
```

```
    print(numero)
```

```
    numero -= 1
```

- **Funciones de evaluación:**

```
def es_positivo( n):
```

```
    return n > 0
```

```
print(es_positivo(5))
```

```
print(es_positivo(-2))
```

Compresión de listas

Requisitos de finalización

La **compresión de listas** es una técnica en Python para crear [listas](#) de manera concisa y legible. Es una forma rápida de generar una nueva lista aplicando una expresión a cada elemento de una secuencia o iterable, opcionalmente filtrando elementos con una condición. Este enfoque no solo hace que el código sea más corto, sino que también puede ser más eficiente en términos de rendimiento en comparación con los bucles tradicionales.

Sintaxis de la Compresión de [Listas](#)

La sintaxis básica de una compresión de [listas](#) es la siguiente:

[expresion [for](#) elemento [in](#) iterable [if](#) condicion]

- **expresion:** Una expresión para calcular o transformar cada elemento.
- **elemento:** Cada elemento del [iterable](#) (por ejemplo, una lista, rango, cadena).
- **iterable:** La secuencia o colección que se está recorriendo.
- **condicion** (opcional): Un filtro que selecciona elementos del [iterable](#).

Ejemplos:

- **Crear una lista de números:**

```
numeros = [n for n in range(5)]
```

```
print(numeros)
```

- **Elevar al cuadrado cada número en una lista:**

```
cuadrados = [n**2 for n in range(6)]
```

```
print(cuadrados)
```

- **Filtrar elementos con una condición:**

```
pares = [n for n in range(10) if n % 2 == 0]
```

```
print(pares)
```

- **Convertir una lista de cadenas a mayúsculas:**

```
palabras = ["python", "es", "genial"]
```

```
mayusculas = [palabra.upper() for palabra in palabras]
```

```
print(mayusculas)
```

- **Usar una expresión condicional en la compresión de [listas](#):**

```
resultado = [x if x % 2 == 0 else "impar" for x in range(5)]
```

```
print(resultado)
```

Conversión de variables

Requisitos de finalización

La conversión de tipos de variables permite cambiar el tipo de dato de una variable a otro, por ejemplo, de un entero a un flotante o de una cadena a un entero. Esto es útil cuando necesitas realizar operaciones matemáticas con números que originalmente fueron ingresados como cadenas de texto, o cuando deseas convertir un valor a un tipo específico para cumplir con los requisitos de una función.

Ejemplo de Conversión de Tipos:

```
num1 = 20
```

```
num2 = 30.5
```

```
print(type(num1))
```

```
print(type(num2))
```

Conversión de un flotante a entero:

```
num2 = int(num1)
```

```
print(num2)
```

```
print(type(num2))
```

Conversión de una String a entero:

```
edad = input("Dime tu edad: ")
```

```
print(type(edad))
```

```
edad = int(edad)
```

```
print(type(edad))
```

Crear y Escribir Archivos

Requisitos de finalización

Abrir un archivo para escritura:

- La función `open()` para abrir un archivo llamado `prueba.txt`.
- El modo `"a"` indica que se está abriendo el archivo en modo de **anexado** (append), lo que significa que el nuevo contenido se añadirá al final del archivo si ya existe, o se creará un nuevo archivo si no existe.

```
archivo = open("prueba.txt", "a")
```

Escribir en el archivo con `write()`:

- `write()` escribe cadenas directamente en el archivo.
- `\n` es un carácter de nueva línea.

```
archivo.write("Mi\n")
```

```
archivo.write("Nombre")
```

Escribir múltiples líneas a la vez:

Se utilizan comillas triples (`"""`) para escribir múltiples líneas en una sola llamada `write()`. Esto añade el texto exactamente como se presenta, incluyendo saltos de línea.

```
archivo.write("""
```

```
Juan
```

```
Gonzalez""")
```

Escribir una lista de cadenas sin saltos de línea automáticos:

`writelines()` toma una lista de cadenas y las escribe en el archivo una tras otra, sin añadir saltos de línea entre ellas automáticamente.

```
archivo.writelines(['hola', 'mundo', 'soy', 'un', 'estudiante'])
```

Escribir líneas usando un bucle:

Este bucle recorre cada elemento en la lista `lista` y usa `writelines()` para escribir cada elemento en el archivo, seguido de un salto de línea.

```
lista = ['hola', 'mundo', 'soy', 'un', 'estudiante']
```

```
for p in lista:
```

```
    archivo.writelines(p + "\n")
```

Escribir texto adicional:

Esta línea añade la palabra "Bienvenido" al final del archivo.

```
archivo.write("Bienvenido")
```

Cerrar el archivo:

Es importante cerrar el archivo para asegurarse de que todos los cambios se guarden correctamente. `close()` libera los recursos del sistema asociados con el archivo.

Consideraciones:

- **Uso de "a" en `open()`:** El modo "a" es útil si quieres añadir contenido a un archivo existente sin sobrescribir su contenido. Si quisieras sobrescribirlo, usarías el modo "w" (write).
- **Métodos `write()` vs `writelines()`:** `write()` se usa para escribir una sola cadena al archivo, mientras que `writelines()` se usa para escribir una lista de cadenas.

Uso de *with*:

Es recomendable utilizar un bloque `with` para manejar archivos, ya que asegura que el archivo se cierre automáticamente incluso si ocurre una excepción:

```
with open("prueba.txt", "a") as archivo:
```

```
    archivo.write("Mi\n")
```

```
    archivo.write("Nombre")
```

Funciones

Requisitos de finalización

`def` es la palabra clave en Python para definir una función:

`saludar_persona` es el nombre de la función. Elegir un nombre descriptivo es una buena práctica para indicar lo que hace la función.

`(nombre)` es el parámetro de la función. Este es un valor que se pasa a la función cuando se llama. En este caso, `nombre` es el nombre de la persona a saludar.

La cadena de texto entre triple comillas (""") es un **docstring**. Es una descripción de lo que hace la función y es una buena práctica incluirla, ya que ayuda a entender el propósito de la función. Los **docstrings** también se pueden utilizar para la documentación automática.

```
def saludar_persona(nombre):  
  
    """  
  
    Esta función sirve para saludar  
  
    a las personas  
  
    """  
  
    print(f"Hola {nombre}")
```

Cuerpo de la Función:

```
print(f"Hola {nombre}")
```

Esta línea imprime un saludo personalizado usando una **f-string** (f"..."). Las f-strings permiten incluir expresiones y variables dentro de cadenas de texto de manera sencilla. Aquí, {nombre} se reemplaza por el valor del parámetro nombre cuando la función es llamada.

Llamadas a la Función:

```
saludar_persona("Jose")
```

```
saludar_persona("Marcos")
```

- Estas líneas llaman a la función `saludar_persona` con diferentes argumentos: "Jose" y "Marcos". El valor pasado se asigna al parámetro `nombre` dentro de la función y se usa para generar el saludo.
- Al llamar `saludar_persona("Jose")`, la salida será: `Hola Jose`.
- Al llamar `saludar_persona("Marcos")`, la salida será: `Hola Marcos`.

Explicación de Conceptos Clave:

- **Funciones:** Las funciones permiten organizar el código en módulos lógicos y reutilizables. Facilitan la lectura, el mantenimiento y la depuración del código.
- **Parámetros y Argumentos:** Los parámetros son variables que se definen en la función y sirven como contenedores para los datos de entrada. Los argumentos son los valores reales que se pasan a los parámetros cuando se llama a la función.
- **Docstrings:** Son comentarios descriptivos que documentan el propósito de la función. Son útiles para la documentación del código y se pueden acceder mediante la función `help()` o desde un entorno de desarrollo que soporte documentación integrada.
- **f-strings:** Son una forma moderna y eficiente de [formatear cadenas](#) en Python. Usando `f"texto {variable}"`, se puede insertar el valor de la variable dentro de la cadena de forma sencilla.

Decisiones

Requisitos de finalización

Decisiones basadas en match de String:

```
mascota = "perro"
```

```
if mascota == "gato":
```

```
    print("Tienes un gato")
```

```
elif mascota == "perro":
```

```
    print("Tienes un perro")
```

```
elif mascota == "pez":
```

```
    print("Tienes un pez")
```

```
else:
```

```
    print("No sé qué animal tienes")
```

- **Definición de la Variable:** `mascota` se asigna con el valor `"perro"`.
- **Condicional `if`:** Comprueba si el valor de `mascota` es `"gato"`. Si es cierto, imprime "Tienes un gato".
- **Condicional `elif`:** Se utiliza para comprobar otras condiciones si la anterior no se cumple. Aquí, comprueba si `mascota` es `"perro"`, y como es cierto, imprime "Tienes un perro".
- **Otro `elif`:** Comprueba si `mascota` es `"pez"`. Este bloque no se ejecuta porque `mascota` es `"perro"`.
- **Condicional `else`:** Si ninguna de las condiciones anteriores se cumple, ejecuta el bloque `else`. En este caso, no se ejecuta porque la condición `mascota == "perro"` es verdadera.

Decisiones Anidadas Basadas en `edad` y `calificacion`:

```
edad = 16
```

```
calificacion = 9
```

```
if edad < 18:
```

```
    print("Eres menor de edad")
```

```
    if calificacion >= 7:
```

```
        print("Aprobado")
```

```
    else:
```

```
print("No aprobado")
```

else:

```
print("Eres adulto")
```

- **Condicional if Principal:** Comprueba si `edad` es menor que 18. Como `edad` es 16, esta condición se cumple y ejecuta el bloque asociado.
 - Imprime "Eres menor de edad".
- **Condicional Anidado:** Dentro del bloque `if` principal, hay otro `if` que comprueba si `calificacion` es mayor o igual a 7. Como `calificacion` es 9, esta condición es verdadera.
 - Imprime "Aprobado".
- Si `calificacion` fuera menor que 7, ejecutaría el bloque `else` asociado, imprimiendo "No aprobado".
- Si `edad` no fuera menor que 18, el [programa](#) ejecutaría el bloque `else` del primer `if`, imprimiendo "Eres adulto".

Diccionarios

Requisitos de finalización

Los diccionarios son estructuras de datos que almacenan pares clave-valor. Son mutables, lo que significa que se pueden modificar después de su creación.

Se accede a los valores usando la clave correspondiente dentro de corchetes (`[]`). Puedes cambiar el valor asociado a una clave existente o añadir nuevos pares clave-valor asignando un valor a una nueva clave.

Son útiles para almacenar y manipular datos de forma estructurada, lo que los hace esenciales en muchas aplicaciones de programación.

Métodos útiles: `keys()`, `values()` y `items()` son [métodos](#) que permiten acceder a las claves, los valores y los pares clave-valor de un diccionario, respectivamente.

Creación de un Diccionario Simple:

```
diccionario = {"c1": "valor1", "c2": "valor2"}
```

```
print(type(diccionario))
```

```
print(diccionario)
```

- Aquí, `diccionario` es un diccionario con dos pares clave-valor: `"c1": "valor1"` y `"c2": "valor2"`.
- `type(diccionario)` devuelve `<class 'dict'>`, indicando que `diccionario` es de tipo diccionario.
- Al imprimir `diccionario`, se muestra el contenido completo del diccionario.

Acceder a un Valor Usando una Clave:

```
resultado = diccionario["c1"]
```

```
print(resultado)
```

Se accede al valor asociado con la clave "c1" usando la notación de corchetes: `diccionario["c1"]`. Esto devuelve el valor "valor1".

Diccionario con Información de un Cliente:

```
cliente = {
```

```
    "nombre": "Jose",
```

```
    "apellido": "Fulano",
```

```
    "peso": "71",
```

```
    "altura": "1.81"
```

```
}
```

```
consulta = cliente["nombre"]
```

```
print(consulta)
```

- `cliente` es un diccionario con claves "nombre", "apellido", "peso" y "altura", cada una asociada con un valor correspondiente.
- `cliente["nombre"]` devuelve "Jose", el valor asociado con la clave "nombre".

Diccionario con Listas y Diccionarios Anidados:

```
diccionario2 = {"c1": 55, "c2": [10, 20, 30], "c3": {"s1": 100, "s2": 200}}
```

```
print(diccionario2["c3"]["s2"])
```

- `diccionario2` contiene una mezcla de tipos de datos: un número entero, una lista y otro diccionario.
- Para acceder a un valor en un diccionario anidado, se utiliza `diccionario2["c3"]["s2"]`, que devuelve 200.

Acceso y Modificación de Valores en una Lista dentro de un Diccionario:

```
diccionario3 = {"c1": ["a", "b", "c"], "c2": ["d", "e", "f"]}
```

```
print((diccionario3["c2"][1]).upper())
```

- `diccionario3` contiene listas como valores.
- `diccionario3["c2"][1]` accede al segundo elemento de la lista asociada con "c2", que es "e".
- El método `upper()` convierte "e" a mayúscula, dando como resultado "E".

Añadir y Modificar Elementos en un Diccionario:

```
diccionario4 = {1: "a", 2: "b"}  
print(diccionario4)
```

```
diccionario4[3] = "c"  
print(diccionario4)
```

```
diccionario4[2] = "B"  
print(diccionario4)
```

- `diccionario4` inicialmente contiene dos pares clave-valor: 1: "a" y 2: "b".
- `diccionario4[3] = "c"` añade un nuevo par clave-valor 3: "c".
- `diccionario4[2] = "B"` modifica el valor asociado con la clave 2 de "b" a "B".

Obtener Claves, Valores y Pares Clave-Valor:

```
print(diccionario4.keys())
```

```
print(diccionario4.values())
```

```
print(diccionario4.items())
```

- `diccionario4.keys()` devuelve una vista de todas las claves en el diccionario: [1, 2, 3].
- `diccionario4.values()` devuelve una vista de todos los valores en el diccionario: ['a', 'B', 'c'].
- `diccionario4.items()` devuelve una vista de todos los pares clave-valor en el diccionario: [(1, 'a'), (2, 'B'), (3, 'c')].

Directorios

Requisitos de finalización

Python permite manipular rutas de archivos, crear y eliminar directorios, y acceder a los contenidos de los archivos.

El módulo `os` ofrece [funciones](#) para interactuar con el sistema operativo, como cambiar directorios, crear y eliminar archivos y directorios, y manipular rutas de archivos. Por su parte, el módulo `pathlib` proporciona una interfaz moderna y orientada a objetos para trabajar con rutas de archivos y directorios, facilitando operaciones complejas y proporcionando una mayor legibilidad del código.

Módulos necesarios:

```
import os
```

```
from pathlib import Path, PureWindowsPath
```

- `os`: Proporciona una forma de interactuar con el sistema operativo. Incluye [funciones](#) para trabajar con archivos, directorios y otros elementos del sistema.

- `pathlib`: Ofrece una forma orientada a objetos para manejar rutas de archivos y directorios. Es más moderna y fácil de usar que algunas de las [funciones](#) del módulo `os`.

Obtención de la Ruta Actual del Directorio de Trabajo:

```
ruta = os.getcwd()
```

```
print(ruta)
```

`os.getcwd()`: Devuelve la ruta actual del directorio de trabajo en formato de bytes. Si prefieres una cadena de texto, puedes usar `os.getcwd()`.

Cambio de Directorio de Trabajo:

```
os.chdir("nueva_ruta")
```

`os.chdir()`: Cambia el directorio de trabajo actual a la ruta especificada. Esto es útil cuando necesitas operar en un directorio específico.

Creación de un Nuevo Directorio:

```
os.makedirs("ruta_nuevo_directorio")
```

`os.makedirs()`: Crea un nuevo directorio en la ruta especificada, incluyendo cualquier directorio intermedio necesario.

Manipulación de Rutas de Archivos:

```
ruta = "ruta_del_directorio"
```

```
elemento = os.path.basename(ruta)
```

```
elemento = os.path.dirname(ruta)
```

```
elemento = os.path.split(ruta)
```

- `os.path.basename(ruta)`: Devuelve el nombre del archivo o del último componente de la ruta.
- `os.path.dirname(ruta)`: Devuelve la parte del directorio de la ruta.
- `os.path.split(ruta)`: Devuelve una [tupla](#) que contiene el directorio y el nombre del archivo.

Eliminación de un Directorio:

```
os.rmdir("ruta_del_directorio")
```

`os.rmdir()`: Elimina el directorio especificado. Este método solo funciona si el directorio está vacío.

Leer Archivos con Rutas Absolutas y Relativas:

```
archivo = open("prueba.txt")
```

```
print(archivo.read())
```

```
otra_ruta = open("direccion\\prueba.txt")
```

```
print(otra_ruta.read())
```

Usando `open()`, puedes abrir un archivo para leer su contenido. `open("prueba.txt")` abre el archivo en el directorio actual, mientras que una ruta absoluta abre el archivo especificado por esa ruta.

Verificación de Existencia de Archivos:

```
if not carpeta.exists():
```

```
    print("Este archivo no existe")
```

```
else:
```

```
    print("Este archivo existe")
```

`Path.exists()`: Comprueba si el archivo o directorio especificado existe en el sistema de archivos.

Enumerate

Requisitos de finalización

`Enumerate` es una práctica común en Python cuando necesitas tanto los índices como los valores de una lista u otra secuencia. Esto simplifica el código, mejora su legibilidad y reduce la probabilidad de errores. En tareas de procesamiento de [listas](#), bucles sobre datos o creación de estructuras de datos que requieren índices y valores.

La **Función** `enumerate` es una función integrada en Python que toma una colección (como una lista, [tupla](#) o cadena) y devuelve un objeto iterador que produce tuplas. Cada [tupla](#) contiene un índice y el elemento correspondiente de la colección.

Iteración Manual con un Contador:

```
lista = ['a', 'b', 'c']
```

```
indice = 0
```

```
for item in lista:
```

```
    print(indice, item)
```

```
    indice += 1
```

- En este ejemplo, una lista llamada `lista` contiene los elementos `['a', 'b', 'c']`.
- Se utiliza una variable `indice` para rastrear la posición del elemento en la lista.
- En cada iteración del bucle `for`, se imprime el valor de `indice` y el `item` actual de la lista, y luego se incrementa `indice` manualmente.

Donde la salida es:

0 a

1 b

2 c

Uso de `enumerate` para Simplificar la Iteración:

```
for indice, item in enumerate(range(50, 55)):
```

```
    print(indice, item)
```

- En este ejemplo, se usa `enumerate` para iterar sobre una secuencia generada por `range(50, 55)`, que produce los números del 50 al 54.
- `enumerate` devuelve tanto el índice como el valor del elemento en cada iteración, lo que elimina la necesidad de manejar manualmente el índice.

Donde la salida es:

0 50

1 51

2 52

3 53

4 54

Convertir una Lista Enumerada en Tuplas:

```
mi_tupla = list(enumerate(lista))
```

```
print(mi_tupla)
```

- `list(enumerate(lista))` convierte la enumeración de `lista` en una lista de tuplas, donde cada `tupla` contiene un par índice-valor.
- El resultado es una lista de tuplas: `[(0, 'a'), (1, 'b'), (2, 'c')]`.

Donde la salida es:

```
[(0, 'a'), (1, 'b'), (2, 'c')]
```

For

Requisitos de finalización

Un bucle `for` se utiliza para iterar sobre una secuencia (como una lista, [tupla](#), diccionario, conjunto o cadena de caracteres) y ejecutar un bloque de código para cada elemento de esa secuencia. Es una forma de automatizar tareas repetitivas, ejecutando el mismo bloque de código varias veces, pero con diferentes datos.

Iterar sobre una lista y obtener el índice de cada elemento

```
lista = ['a', 'b', 'c']

for l in lista:
    numero_letra = lista.index(l) + 1
    print(f"Letra {numero_letra}: {l}")
```

Este bucle itera sobre la lista `lista`, obtiene el índice de cada elemento utilizando `lista.index(l)` y lo incrementa en 1 para empezar desde 1 en lugar de 0. Luego imprime el número y la letra.

Iterar sobre una lista y encontrar elementos que comienzan con una letra específica

```
lista2 = ["pablo", "laura", "fedede", "luis", "julia"]

for nombre in lista2:
    if nombre.startswith("l"):
        print(nombre)
```

En este caso el bucle itera sobre la lista `lista2` y utiliza el método `startswith("l")` para comprobar si el nombre empieza con la letra "l". Si es así, imprime el nombre.

Sumar elementos de una lista

```
numeros = [1, 2, 3, 4, 5]
mi_valor = 0

for numero in numeros:
    mi_valor = mi_valor + numero
    print(mi_valor)
```

Este bucle suma los números en la lista `numeros` y almacena el resultado en la variable `mi_valor`. Finalmente, imprime el valor total.

Iterar sobre una cadena y imprimir cada carácter

```
for letra in "python":  
    print(letra)
```

Desempaquetar listas de pares

```
for a, b in [[1, 2], [3, 4], [5, 6]]:  
    print(a)  
    print(b)
```

Iterar sobre un diccionario y acceder a claves y valores

```
dic = {"clave1": "a", "clave2": "b", "clave3": "c"}
```

```
for a, b in dic.items():  
    print(a, b)
```

Este bucle itera sobre los elementos de un diccionario `dic`. El método `items()` devuelve un objeto de vista que muestra una lista de pares clave-valor. Cada par se descompone en `a` (clave) y `b` (valor), y luego se imprime.

Formatear cadenas

Requisitos de finalización

Es una técnica común para crear cadenas de texto que incluyen valores de variables, y hay varias formas de hacerlo. En tu ejemplo, utilizas tres métodos diferentes para formatear cadenas, cada uno con sus ventajas.

Concatenación usando el operador +

```
x = 10  
  
y = 5  
  
print("Mis numeros son: " + str(x) + " y " + str(y))
```

Aquí, se usa el operador `+` para concatenar cadenas y se convierte cada número a una cadena usando `str()`.

Esto es simple pero puede volverse complicado y menos legible cuando tienes que concatenar muchos valores.

Método `str.format()`

```
print("Mis numeros son: {} y {}".format(x, y))  
  
print("La suma de {} y {} es igual a {}".format(x, y, x + y))
```

El método `str.format()` permite insertar variables en posiciones específicas usando `{}` como marcadores de posición. Las variables se pasan al método `format()` en el orden correspondiente a los marcadores. Es más legible y manejable cuando se trabaja con múltiples variables.

F-Strings (Formatted String Literals)

```
color = "rojo"  
matricula = 541926  
  
print(f"El auto es {color} y su matricula es: {matricula}")
```

Las f-strings son una forma moderna y muy eficiente de formatear cadenas (disponible desde Python 3.6). Se colocan las variables directamente dentro de las llaves `{}` precedidas por una `f` antes de las comillas de la cadena. Son rápidas, concisas y fáciles de leer.

Fragmentar cadenas

Requisitos de finalización

Fragmentar cadenas en Python es una técnica poderosa que permite extraer partes específicas de una cadena, lo cual es útil para manipulación y análisis de texto. Se hace utilizando la notación de **slicing** (rebanado) con la sintaxis `[inicio:fin:paso]`.

Dada la cadena:

```
texto = "ABCDEFGHJKLM"
```

Obtener un solo carácter:

```
fragmento = texto[2]
```

Fragmento desde una posición inicial a una final:

```
fragmento = texto[2:5]
```

Fragmento desde una posición inicial hasta el final:

```
fragmento = texto[2:]
```

Fragmento desde el inicio hasta una posición final:

```
fragmento = texto[:5]
```

Fragmento con un paso específico:

```
fragmento = texto[2:10:2]
```

Fragmento con un paso omitiendo inicio y fin:

```
fragmento = texto[:,3]
```

Invertir la cadena:

```
fragmento = texto[::-1]
```

Funciones dinámicas

Requisitos de finalización

Las **funciones dinámicas** se refieren a [funciones](#) que se pueden crear, modificar, y pasar como argumentos durante la ejecución del [programa](#). Estas [funciones](#) se pueden manipular como cualquier otro objeto (por ejemplo, [listas](#), [diccionarios](#)), lo que permite una gran flexibilidad y dinamismo en la programación.

En el siguiente código, la función `chequear_3_cifras` está diseñada para recibir una lista de números y devolver una nueva lista que solo contiene aquellos números que tienen exactamente tres cifras, es decir, que están en el rango de 100 a 999:

```
def chequear_3_cifras(lista):
```

```
    lista_3 = []
```

```
for n in lista:
```

```
    if n in range(100, 1000):
```

```
        lista_3.append( n)
```

```
    else:
```

```
        pass
```

```
return lista_3
```

```
resultado = chequear_3_cifras([555, 99, 600])  
print(resultado)
```

Lógica del código:

- **Definición de la función `chequear_3_cifras`:**
 - La función toma un argumento `lista`, que es una lista de números.
 - Crea una lista vacía `lista_3` para almacenar los números de tres cifras.
- **Iterar sobre cada número en la lista:**
 - Se utiliza un bucle `for` para recorrer cada número `n` en la lista de entrada.
 - Se verifica si el número está en el rango de 100 a 999 usando `n in range(100, 1000)`. La función `range(100, 1000)` genera números del 100 al 999 (el límite superior 1000 no está incluido).
- **Añadir números de tres cifras a la lista:**
 - Si el número tiene tres cifras, se añade a `lista_3` usando `lista_3.append( n)`.
 - Si no, se usa `pass` para no hacer nada (este `else` es en realidad innecesario y puede omitirse para simplificar el código).
- **Devolver la lista filtrada:**
 - Finalmente, la función devuelve la lista `lista_3` que contiene solo números de tres cifras.

Index()

Requisitos de finalización

El método `index()` en Python se utiliza para encontrar la posición de un carácter o subcadena dentro de una cadena de texto. Si el carácter o subcadena se encuentra en la cadena, `index()` devuelve la posición del primer carácter encontrado. Si no se encuentra, produce un error `ValueError`. Además, existe un método llamado `rindex()` que busca la última aparición de un carácter o subcadena.

```
mi_texto = "Esta es una prueba"
```

Obtener el primer carácter

```
resultado = mi_texto[0]
```

Obtener el cuarto carácter desde el final

```
resultado = mi_texto[-4]
```

Encontrar la posición de la letra 'n'

```
resultado = mi_texto.index("n")
```

Encontrar la posición de la subcadena "prueba"

```
resultado = mi_texto.index("prueba")
```

Encontrar la letra 'a' entre las posiciones 5 y 11

```
resultado = mi_texto.index("a", 5, 11)
```

Encontrar la última aparición de la letra 'a'

```
resultado = mi_texto.rindex("a")
```

****kwargs**

Requisitos de finalización

En Python, ****kwargs** es una forma de pasar un número variable de argumentos con nombre (argumentos clave-valor) a una función. **kwargs** significa **keyword arguments** y se usa comúnmente cuando no sabes de antemano cuántos argumentos se pasarán a la función.

Ejemplo:

```
def suma(**kwargs):
```

```
    total = 0
```

```
    for clave, valor in kwargs.items():
```

```
        print(f"{clave} = {valor}")
```

```
total += valor
```

```
return total
```

```
print(suma(x=3, y=5, z=2))
```

`kwargs` es un diccionario que contiene todos los argumentos pasados a la función con nombre.

`kwargs.items()` devuelve pares clave-valor, que se recorren en el bucle `for`.

En el ejemplo, `x=3`, `y=5`, `z=2` son los argumentos con nombre. El total de la suma es $3 + 5 + 2 = 10$.

Limpiar consola

Requisitos de finalización

La función `system("cls")` del módulo `os` sirve para limpiar la consola en sistemas Windows. La función `system("cls")` envía un comando al sistema operativo para ejecutar el comando `cls`, que limpia la pantalla de la consola de Windows.

`from os import system` importa solo la función `system` del módulo `os`. Esta función permite ejecutar comandos del sistema operativo desde un script de Python.

Ejemplo:

```
from os import system
```

```
nombre = input("Dime tu nombre: ")
```

```
edad = input("Dime tu edad: ")
```

```
system("cls")
```

```
print(f"Tu nombre es {nombre} y tienes {edad} años")
```

Listas

Requisitos de finalización

En Python, las listas son una estructura de datos fundamental que permite almacenar una colección ordenada de elementos. Puedes realizar diversas operaciones con listas, como acceder a elementos, modificar valores, combinar listas.

```
mi_lista = ["a", "b", "c"]
```

```
mi_lista2 = ["d", "e", "f"]
```

```
otra_lista = ["hola", 55, 6.1]
```

Longitud de una lista con `len()`:

```
resultado = len(mi_lista)
```

Acceso a elementos con slicing:

```
resultado = mi_lista[0:]
```

Concatenar listas:

```
mi_lista3 = mi_lista + mi_lista2
```

Modificar Elementos en una Lista

```
mi_lista3[0] = "alfa"
```

Añadir un elemento al final de la lista con `append()`:

```
mi_lista3.append("g")
```

Eliminar un elemento por su índice con `pop()`:

```
eliminado = mi_lista3.pop(0)
```

Ordenar la lista de forma ascendente:

```
lista = ["g", "o", "b", "m", "c"]
```

```
lista.sort()
```


Invertir el orden de la lista:

```
lista.reverse()
```

Manipular archivos

Requisitos de finalización

Para manipular archivos en Python, debes seguir unos pasos básicos para abrir, leer y cerrar los archivos.

- **Abrir un archivo:** Usa la función `open()` para abrir un archivo. Puedes especificar el modo en el que quieres abrirlo ('r' para lectura, 'w' para escritura, 'a' para agregar contenido).
- **Leer el archivo:** Una vez que el archivo está abierto, puedes leer su contenido de diferentes maneras:
 - `read()`: Lee todo el contenido del archivo en una sola cadena.
 - `readline()`: Lee una línea a la vez.
 - `readlines()`: Lee todas las líneas y las devuelve como una lista.
- **Cerrar el archivo:** Siempre es una buena práctica cerrar el archivo con `close()` después de terminar de trabajar con él.

Ejemplo:

Abrir el archivo en modo lectura:

```
mi_archivo = open("prueba.txt")
```

Leer todo el contenido del archivo:

```
texto = mi_archivo.read()
```

```
print(texto)
```

Leer una línea del archivo:

```
primer_linea = mi_archivo.readline()
```

```
print(primer_linea.upper())
```

Leer la siguiente línea del archivo y eliminar el salto de línea:

```
primer_linea = mi_archivo.readline()
```

```
print(primer_linea.rstrip())
```

Leer la siguiente línea del archivo:

```
primer_linea = mi_archivo.readline()

print(primer_linea)
```

Leer todas las líneas restantes y almacenarlas en una lista:

```
todas = mi_archivo.readlines()

print(todas)
```

Extraer y eliminar la última línea de la lista:

```
todas = todas.pop()

print(todas)
```

Cerrar el archivo:

```
mi_archivo.close()
```

Alternativa:

```
with open("prueba.txt", "r") as mi_archivo:
    texto = mi_archivo.read()
    print(texto)
    mi_archivo.close()
```

Match

Requisitos de finalización

La declaración `match` en Python, introducida en la versión 3.10, ofrece una forma más expresiva y legible de hacer coincidencias y descomponer estructuras de datos en comparación con los tradicionales `if-elif-else`.

Versión anterior (sin **match**):

```
serie = "N-02"

if serie == "N-01":

    print("Samsung")

elif serie == "N-02":

    print("Nokia")

elif serie == "N-03":

    print("Motorola")

else:

    print("No existe ese producto")
```

Versión nueva (con **match**):

```
serie = "N-02"

match serie:

    case "N-01":

        print("Samsung")

    case "N-02":

        print("Nokia")

    case "N-03":

        print("Motorola")

    case _:

        print("No existe ese producto")
```

Métodos

Requisitos de finalización

En Python, los métodos de cadena (**str**) permiten manipular y transformar texto de diversas maneras.

Métodos de Cadena en Python:

- **upper()**: Convierte todos los caracteres de la cadena a mayúsculas.
- resultado = texto.upper()
- **lower()**: Convierte todos los caracteres de la cadena a minúsculas.
- resultado = texto.lower()
- **split(sep)**: Divide la cadena en una lista de subcadenas basadas en el delimitador **sep**. Si no se proporciona **sep**, se usa cualquier espacio en blanco como delimitador.

```
resultado = texto.split()
```

```
resultado = texto.split("t")
```

- **find(sub)**: Encuentra la primera ocurrencia del subcadena **sub** y devuelve su índice. Retorna **-1** si no se encuentra.
- resultado = texto.find("texto")
- **replace(old, new)**: Reemplaza todas las ocurrencias de la subcadena **old** con la subcadena **new**.
- resultado = texto.replace("Jose", "Todos")
- resultado = texto.replace("e", "x")
- **join(iterable)**: Une una secuencia de cadenas con la cadena actual como separador.
- a = "Aprender"
- b = "Python"
- c = "es"
- d = "genial"
- e = " ".join([a, b, c, d])

Los [métodos](#) **min()** y **max()** en Python se utilizan para encontrar el valor mínimo y máximo en un conjunto de valores, ya sea en una lista, una [tupla](#) o incluso un diccionario.

- **En Conjuntos de Números:**

min() y **max()** encuentran el valor mínimo y máximo, respectivamente, entre los números proporcionados.

```
menor = min(58, 96, 72, 64, 35)
mayor = max(58, 96, 72, 64, 35)
```

- **En [Listas](#) y Tuplas:**

Puedes usar **min()** y **max()** para encontrar el valor mínimo y máximo en [listas](#) y tuplas.

```
lista = (58, 96, 72, 64, 35)
print(f"El menor es {min(lista)} y el mayor es {max(lista)}")
```

- **En [Listas](#) de Cadenas:**

min() y **max()** también pueden aplicarse a [listas](#) de cadenas, donde la comparación se realiza lexicográficamente (alfabéticamente).

```
nombres = ["Juan", "Pablo", "Alicia", "Carlos"]
print(min(nombres))
```

- **En Cadenas:**

Cuando se usa en una cadena, `min()` y `max()` encuentran el carácter mínimo y máximo basado en el orden lexicográfico.

```
nombre = "Carlos"
print(min(nombre.lower()))
```

- **En Diccionarios:**

Cuando se usa con un diccionario, `min()` y `max()` operan sobre las claves del diccionario si no se especifica otro parámetro. Usando `dic.values()` se opera sobre los valores.

```
dic = {"C1": 45, "C2": 11}
print(min(dic.values()))
```

Path

Requisitos de finalización

La biblioteca `pathlib` en Python proporciona una manera intuitiva y orientada a objetos de manejar rutas y archivos. A continuación te explico los conceptos y `métodos` utilizados en tu código.

Creación de Rutas:

```
from pathlib import Path
```

```
guia = Path("europa", "españa", "Barcelona", "sagrada_familia.txt")
```

`Path()` crea una instancia de un objeto `Path` que representa una ruta en el sistema de archivos.

Modificar el Nombre del Archivo:

```
guia2 = guia.with_name("la_pedreda.txt")
```

```
print(guia2)
```

`with_name(new_name)` cambia el nombre del archivo en la ruta sin modificar el directorio.

Obtener el Directorio Padre:

```
print(guia.parent)
```

`parent` obtiene el directorio padre de la ruta actual.

Buscar Archivos con `glob`:

```
for txt in Path(guia).glob("*.txt"):
```

```
    print(txt)
```

`glob(pattern)` busca archivos que coinciden con el patrón en el directorio actual.

Buscar Archivos Recursivamente:

```
for txt in Path(guia).glob("**/*.txt"):
```

```
    print(txt)
```

`glob("**/*.txt")` busca archivos .txt en todos los subdirectorios de manera recursiva.

Rutas Relativas:

```
guia = Path("europa", "españa", "Barcelona", "sagrada_familia.txt")
```

```
en_auropa = guia.relative_to(Path("Europa"))
```

```
en_espania = guia.relative_to(Path("Europa", "España"))
```

```
print(en_auropa)
```

```
print(en_espania)
```

`relative_to(base_path)` devuelve la ruta relativa con respecto a `base_path`. Si la ruta no comienza con `base_path`, se lanzará una excepción.

Random

Requisitos de finalización

La biblioteca `random` en Python proporciona una serie de [funciones](#) para generar números aleatorios y realizar operaciones aleatorias, como seleccionar elementos al azar de una secuencia o barajar [listas](#).

`randint(a, b):`

Genera un número entero aleatorio entre `a` y `b` (ambos inclusive).

```
aleatorio1 = randint(1, 50)
```

```
print(aleatorio1)
```

`uniform(a, b):`

Genera un número de punto flotante aleatorio entre `a` y `b`. Luego, `round(valor, 1)` redondea el número a un decimal.

```
aleatorio2 = round(uniform(1, 5), 1)
print(aleatorio2)
```

random():

Genera un número de punto flotante aleatorio entre 0.0 y 1.0.

```
aleatorio3 = random()
print(aleatorio3)
```

choice(seq):

Selecciona un elemento aleatorio de la secuencia `seq`.

```
colores = ["azul", "rojo", "verde", "amarillo"]
aleatorio4 = choice(colores)
print(aleatorio4)
```

shuffle(seq):

Baraja los elementos de la lista `seq` en su lugar.

```
numeros = list(range(5, 50, 5))
shuffle(numeros)
print(numeros)
```

Range

Requisitos de finalización

En Python, `range()` es una función muy útil para generar secuencias de números, especialmente cuando se usa en combinación con bucles `for`.

Sintaxis Básica de `range()`:

La función `range(start, stop, step)` genera una secuencia de números comenzando en `start`, deteniéndose antes de `stop`, y avanzando en pasos de tamaño `step`.

Si solo se proporciona un argumento, se toma como el valor `stop`, y `start` por defecto es 0, con `step` por defecto de 1.

```
for numero in range(1, 5, 1):
```

```
    print(numero)
```

En este ejemplo, `range(1, 5, 1)` genera los números 1, 2, 3, 4. El `stop` es 5, pero el rango se detiene antes de alcanzarlo.

Crear una Lista Usando `range()`:

Puedes convertir el objeto `range` a una lista usando `list(range(...))`. Esto es útil para generar [listas](#) de números de forma rápida y eficiente.

```
lista = list(range(1, 101))
```

```
print(lista)
```

Round

Requisitos de finalización

En Python, la función `round()` se utiliza para redondear números a un número específico de decimales o al número entero más cercano.

Redondear al Número Entero Más Cercano:

Cuando se usa `round()` sin un segundo argumento, redondea el número al entero más cercano.

```
resultado = 90 / 7
redondeo = round(resultado)
print(redondeo)
```

En este caso, `resultado` es aproximadamente 12.857, y `round(12.857)` redondea a 13, el entero más cercano.

Redondear a un Número Específico de Decimales:

Puedes especificar cuántos decimales deseas con un segundo argumento en `round()`.

```
valor = round(95.666666666666666666, 3)
print(valor)
print(type(valor))
```

Redondea el número a 3 decimales, dando como resultado 95.667.

Redondeo sin Especificar Decimales:

Redondear un número flotante sin especificar decimales lo convierte en el entero más cercano.

```
valor = round(95.666666666666666666)
print(valor)
print(type(valor))
```

En este caso, 95.666... se redondea a 96, convirtiéndose en un entero.

Tipos de Datos y Redondeo:

El tipo de dato después de redondear puede cambiar dependiendo de cómo se utilice `round()`.

```
valor = 95.666666666666666666
print(round(valor))
print(type(valor))
print(valor)
print(type(valor))
```

Aunque se redondea y se imprime el resultado como un entero, **valor** sigue siendo un número flotante a menos que se asigne explícitamente.

Asignación de Variables:

La asignación de **num2** a **num1** hace que ambas variables apunten al mismo valor.

```
num1 = 10
num2 = 20

num1 = num2
print(num2)
```

Return

Requisitos de finalización

La palabra clave **return** en Python se utiliza en una función para enviar de vuelta un valor al lugar donde se llamó a la función. Es fundamental para obtener resultados de [funciones](#) y seguir usándolos en el resto del [programa](#).

Definición de la Función **multiplicar**:

Esta función toma dos parámetros, **numero1** y **numero2**, y devuelve el producto de estos dos números.

```
def multiplicar(numero1, numero2):

    return numero1 * numero2
```

Aquí, **return numero1 * numero2** calcula el producto de **numero1** y **numero2** y luego devuelve ese valor al lugar donde se llamó la función.

Llamada a la Función y Uso del Valor Devuelto:

Cuando llamas a **multiplicar(5, 10)**, la función se ejecuta con **numero1 = 5** y **numero2 = 10**. La función realiza la multiplicación **5 * 10** y devuelve **50**.

```
resultado = multiplicar(5, 10)
print(resultado)
```

El valor devuelto, 50, se almacena en la variable `resultado` y luego se imprime.

Sets

Requisitos de finalización

Los **sets** en Python son colecciones desordenadas de elementos únicos y mutables, lo que significa que no contienen elementos duplicados y sus elementos pueden ser modificados.

Usando la función `set()`:

```
mi_set = set([1, 2, 3, 4, 5])
print(type(mi_set))
print(mi_set)
```

Aquí, se crea un set a partir de una lista. Los elementos se mantienen únicos y sin orden específico.

Usando Llaves `{}`:

```
mi_set2 = {1, 2, 3, 4, 5, 1, 1, 2, 3, 3, 4}
print(type(mi_set2))
print(mi_set2)
```

Los sets eliminan automáticamente los elementos duplicados. En este caso, solo se mantienen los valores únicos.

Sets con Tuplas:

```
mi_set3 = {1, 2, 3, 4, 5, (1, 2, 3), 1, 1, 2, 3, 3, 4}
print(type(mi_set3))
print(mi_set3)
```

Los sets pueden contener tuplas porque las tuplas son inmutables y, por lo tanto, hashables, pero no pueden contener [listas](#) u otros sets porque no son hashables.

Operaciones:

Tamaño del Set y Comprobación de Elementos:

```
print(len(mi_set))  
print(2 in mi_set)
```

Agregar y Eliminar Elementos:

```
s1 = {1, 2, 3}  
s1.add(4)  
print(s1)
```

```
s1.pop()  
print(s1)
```

```
s1.remove(3)  
print(s1)
```

```
s1.discard(2)  
print(s1)
```

```
s1.clear()  
print(s1)
```

add(): Agrega un elemento al set.

pop(): Elimina un elemento aleatorio. Si el set está vacío, lanza un error.

remove(): Elimina un elemento específico; si no existe, lanza un error.

`discard()`: Elimina un elemento específico sin lanzar error si no existe.

`clear()`: Elimina todos los elementos del set.

Unión de Sets:

```
s2 = {3, 4, 5}
s3 = s1.union(s2)
print(s3)
```

`union()`: Devuelve un nuevo set con todos los elementos de ambos sets sin duplicados.

Restricciones y Características de los Sets

- **Acceso por Índice:** Los sets no soportan indexación. Esto significa que no puedes acceder a sus elementos usando un índice (e.g., `mi_set[0]`). Intentar hacerlo dará un error.
- **Inmutabilidad de los Elementos:** Solo se pueden almacenar objetos inmutables (e.g., números, cadenas, tuplas) dentro de un set. Objetos mutables como [listas](#) o sets no se pueden incluir.

Tupla

Requisitos de finalización

Las **tuplas** en Python son secuencias inmutables de elementos, lo que significa que una vez que se crean, sus valores no se pueden modificar. Las tuplas se utilizan a menudo para agrupar datos heterogéneos y asegurar que los elementos agrupados no sean cambiados accidentalmente.

Creación y Acceso a Tuplas

Definición de Tuplas:

```
mi_tuples = (1, 2, 3, 4)
print(type(mi_tuples))
```

`mi_tuples` es una tupla que contiene cuatro elementos. El uso de paréntesis `()` define la tupla.

```
t = (5, 5.6, 'ff')
print(type(t))
```

`t` es una tupla que contiene un número entero, un número flotante y una cadena.

Acceso a Elementos:

```
print(mi_tuples[0])
```

Las tuplas soportan la indexación, lo que permite acceder a elementos específicos mediante índices.

```
print(mi_tuples)
```

Intentar modificar un elemento de una tupla dará un error de tipo `TypeError`.

Tuplas Anidadas:

```
mi_tuples2 = (1, 2, (10, 20), 4)
print(mi_tuples2[2][0])
```

Las tuplas pueden contener otras tuplas. Aquí se accede al primer elemento de la tupla anidada `(10, 20)`.

Conversión entre [Listas](#) y Tuplas

Puedes convertir una tupla en una lista si necesitas modificarla y luego volver a convertirla en una tupla.

```
mi_tuples2 = list(mi_tuples2)
print(type(mi_tuples2))
```

```
mi_tuples2 = tuple(mi_tuples2)
print(type(mi_tuples2))
```

Aquí, `mi_tuples2` se convierte a una lista y luego de nuevo a una tupla.

Desempaquetado de Tuplas

Asignación Múltiple: Puedes desempacar los elementos de una tupla en variables individuales.

```
x, y, z = t
```

```
print(x, y, z)
```

`t` se desempaqueta en las variables `x`, `y` y `z` según el orden de los elementos en la tupla.

Métodos Útiles para Tuplas

Longitud de una Tupla:

```
mi_tuples3 = (1, 2, 3, 1)
```

```
print(len(mi_tuples3))
```

`len()` devuelve el número de elementos en la tupla.

Contar Ocurrencias de un Elemento:

```
print(mi_tuples3.count(1))
```

`count()` devuelve el número de veces que aparece un elemento en la tupla.

Encontrar el Índice de un Elemento:

```
print(mi_tuples3.index(2))
```

`index()` devuelve la primera posición del elemento especificado en la tupla.

While

Requisitos de finalización

El bucle `while` en Python se utiliza para ejecutar un bloque de código mientras se cumpla una condición específica.

Uso del Bucle `while`

1. Ejemplo Básico de `while`:

```
monedas = 5
```

```
while monedas > 0:
```

```
    print(f"Tengo {monedas} monedas")
```

```
    monedas -= 1
```

```
else:
```

```
    print("No tengo más dinero")
```

- **Explicación:**

- Mientras `monedas` sea mayor que 0, el bucle imprime el número de monedas y luego decrementa el valor de `monedas`.
- Cuando la condición `monedas > 0` ya no se cumple (es decir, `monedas` es 0), el bucle termina y se ejecuta el bloque `else`, que imprime "No tengo más dinero".

2. Bucle `while` con Input del Usuario:

```
respuesta = "s"
```

```
while respuesta == "s":
```

```
    respuesta = input("Quieres seguir? (s/n) ")
```

```
print("¡Gracias!")
```

- **Explicación:**

- El bucle sigue pidiendo al usuario que ingrese "s" para continuar o "n" para detenerse.
- Cuando el usuario ingresa algo distinto de "s", el bucle termina y se imprime "¡Gracias!".

Uso de `break`, `pass` y `continue` en Bucles

1. *break*:

```
nombre = input("Tu nombre: ")
```

```
for letra in nombre:
```

```
    if letra == "r":
```

```
        break
```

```
    print(letra)
```

- **Explicación:**

- El bucle `for` recorre cada letra en `nombre`.
- Si encuentra una letra "r", se usa `break` para salir del bucle.
- Las letras antes de la "r" se imprimen, pero si hay una "r", el bucle se detiene y no se imprimen más letras.

2. *pass*:

```
nombre = input("Tu nombre: ")
```

```
for letra in nombre:
```

```
    if letra == "r":
```

```
        pass
```

```
    print(letra)
```

- **Explicación:**

- El `pass` es un marcador de posición que no realiza ninguna acción.
- Aquí, no se hace nada si la letra es "r", pero se sigue imprimiendo cada letra.

3. *continue*:

```
nombre = input("Tu nombre: ")
```

```
for letra in nombre:
```

```
    if letra == "r":
```

`continue`

`print(letra)`

- **Explicación:**

- El `continue` hace que el bucle pase a la siguiente iteración cuando encuentra una "r".
- Las letras "r" se omiten en la impresión, pero el bucle continúa con las demás letras.

Zip

Requisitos de finalización

La función `zip` en Python se utiliza para combinar varias secuencias (como [listas](#) o tuplas) en una sola, agrupando los elementos correspondientes de cada secuencia en tuplas. Es especialmente útil cuando necesitas iterar sobre múltiples secuencias al mismo tiempo.

Uso de `zip`

Por ejemplo, cuando se combinan tres [listas](#): `nombres`, `edades`, y `cuidades`. Cada lista contiene elementos que corresponden entre sí por índice (es decir, el primer nombre corresponde a la primera edad y a la primera ciudad, y así sucesivamente).

Ejemplo de Uso:

Combinación de [Listas](#) con `zip`:

```
nombres = ["Ana", "Hugo", "Valeria"]
```

```
edades = [65, 29, 42]
```

```
cuidades = ["Lima", "Madrid", "Mexico"]
```

```
combinados = list(zip(nombres, edades, cuidades))
```

```
print(combinados)
```

- **Salida:**

```
[('Ana', 65, 'Lima'), ('Hugo', 29, 'Madrid'), ('Valeria', 42, 'Mexico')]
```

Aquí, `zip` crea una lista de tuplas, donde cada [tupla](#) contiene un nombre, una edad, y una ciudad de las [listas](#) originales.

1. Iteración sobre las Tuplas Combinadas:

`for` nombre, edad, ciudad `in` combinados:

```
print(f"{nombre} tiene {edad} años y vive en {ciudad}")
```

- **Salida:**

Ana tiene 65 años y vive en Lima

Hugo tiene 29 años y vive en Madrid

Valeria tiene 42 años y vive en Mexico

Este bucle descomprime cada [tupla](#) en sus elementos individuales **nombre**, **edad**, y **ciudad**, y luego imprime una cadena formateada para cada conjunto de datos.

Características Adicionales de **zip**

- **Longitud de la salida:** **zip** detiene la combinación en el tamaño de la secuencia más corta. Si una lista tiene más elementos que las otras, esos elementos adicionales se ignorarán.
- **Uso con Diferentes Tipos de Secuencias:** Aunque aquí usamos [listas](#), **zip** funciona con cualquier secuencia iterable, incluyendo tuplas, conjuntos ([sets](#)), y [diccionarios](#) (iterando sobre sus claves, valores, o elementos).